Bootloaders and Device Trees - part 1

Advanced Embedded Software Development

with **Dan Walkes**


University of Colorado **Boulder**

Learning objectives:

Understand the role of the bootloader
Understand the role of Device Trees
Understand simple and more complex bootloader examples.

Bootloader Function

- Initialize the system at a basic level
  - Minimum amount of work necessary to load the kernel.
- Starting with a single CPU core and on-chip memory.
- “Bootstrap” initialize hardware, loading startup software in phases.
- Setting up DRAM
- Setting up storage
- Loading the kernel into RAM

Device Trees

- Before kernel version 3.8 (Feb 2013) platform information was either discovered at runtime or hard-coded into the kernel as platform data.
- Replaced recently by Device Trees (especially on ARM).
- Represents the system as a collection of components in an hierarchy (hence tree name).
- Includes properties in key value pair with form name = “value”

Device Tree Example

```
/dts-v1/;
/{
    model = "TI AM335x BeagleBone";
    compatible = "ti,am33xx";
    #address-cells = <1>;
    #size-cells = <1>;
    cpus {
        #address-cells = <1>;
        #size-cells = <0>;
        cpu@0 {
            compatible = "arm,cortex-a8";
            device_type = "cpu";
            reg = <0>;
        };
    };
    memory@0x80000000 {
        device_type = "memory";
        reg = <0x80000000 0x20000000>; /* 512 MB */
    };
};
```

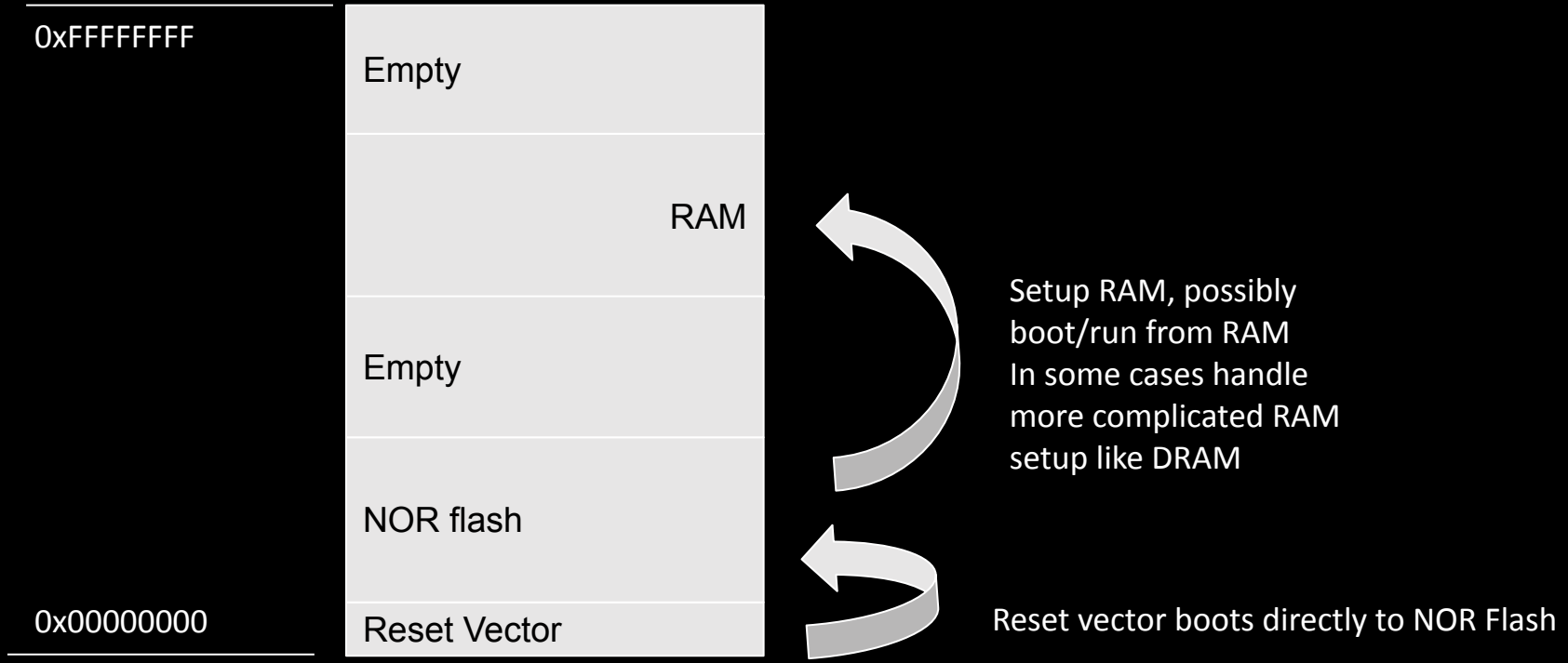
```
/dts-v1/;
{
    intc: interrupt-controller@48200000 {
        compatible = "ti,am33xx-intc";
        interrupt-controller;
        #interrupt-cells = <1>;
        reg = <0x48200000 0x1000>;
    };

    lcdc: lcdc@4830e000 {
        compatible = "ti,am33xx-tilcdc";
        reg = <0x4830e000 0x1000>;
        interrupt-parent = <&intc>;
        interrupts = <36>;
        ti,hwmods = "lcdc";
        status = "disabled";
    };
};
```

Compiling Device Trees

- The ARM bootloader/kernel requires a device tree to be compiled before using
- .dtb - Device Tree Binary or Device Tree Blob
- Compiled with dtc - Device Tree Compiler
- Also known as “Flattened Device Tree” (FDT)

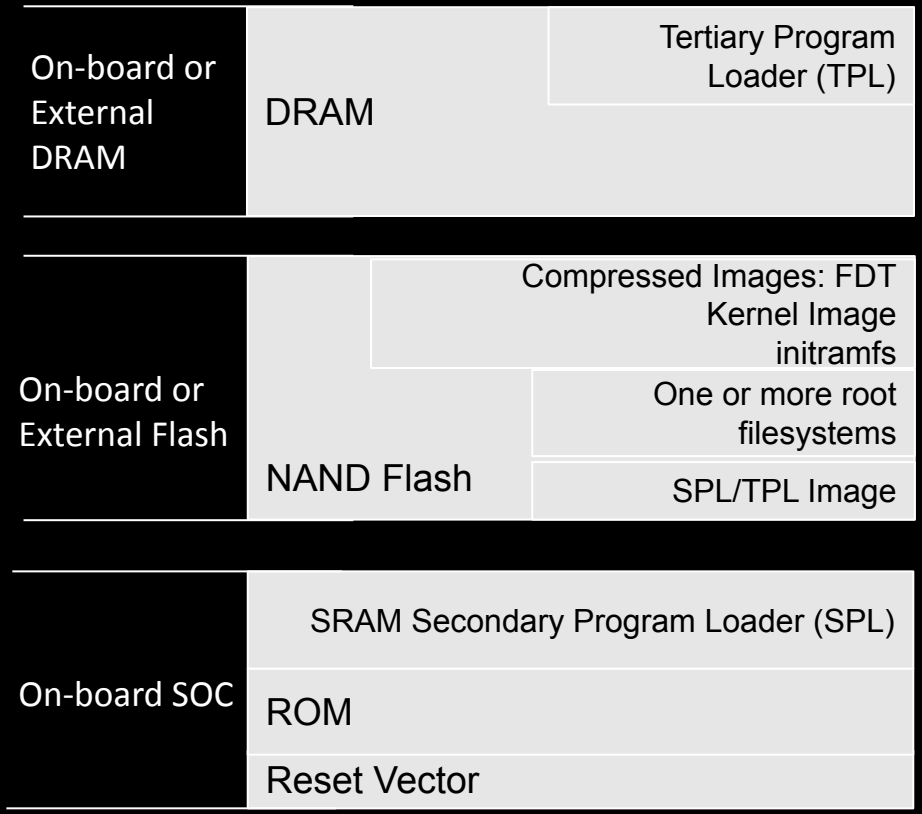
Simple Bootloader Example



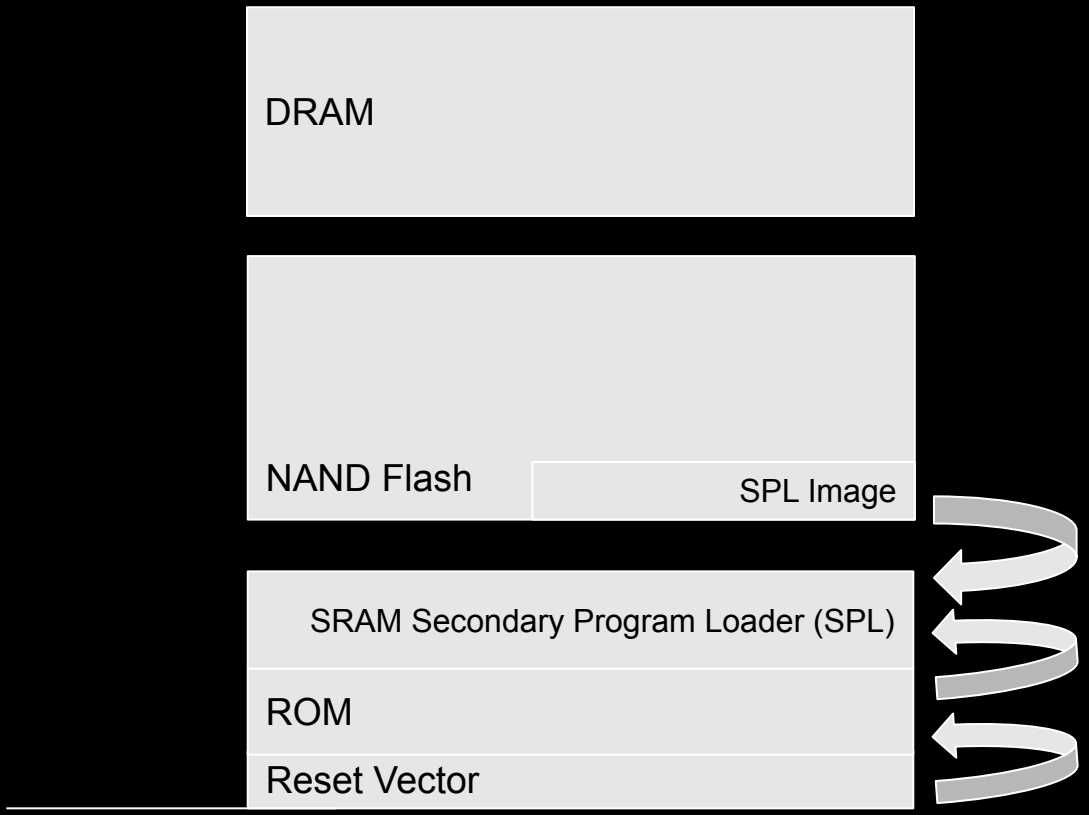
Why Can’t we Use a Simple Bootloader?

- Complexities associated with NAND flash
  - No longer linearly addressible
  - What does this mean?
    - Need ECC, recovery of bad blocks built into the bootloader.
- General complexities of hardware mean more work is needed before we can start Linux.

More Complicated Example



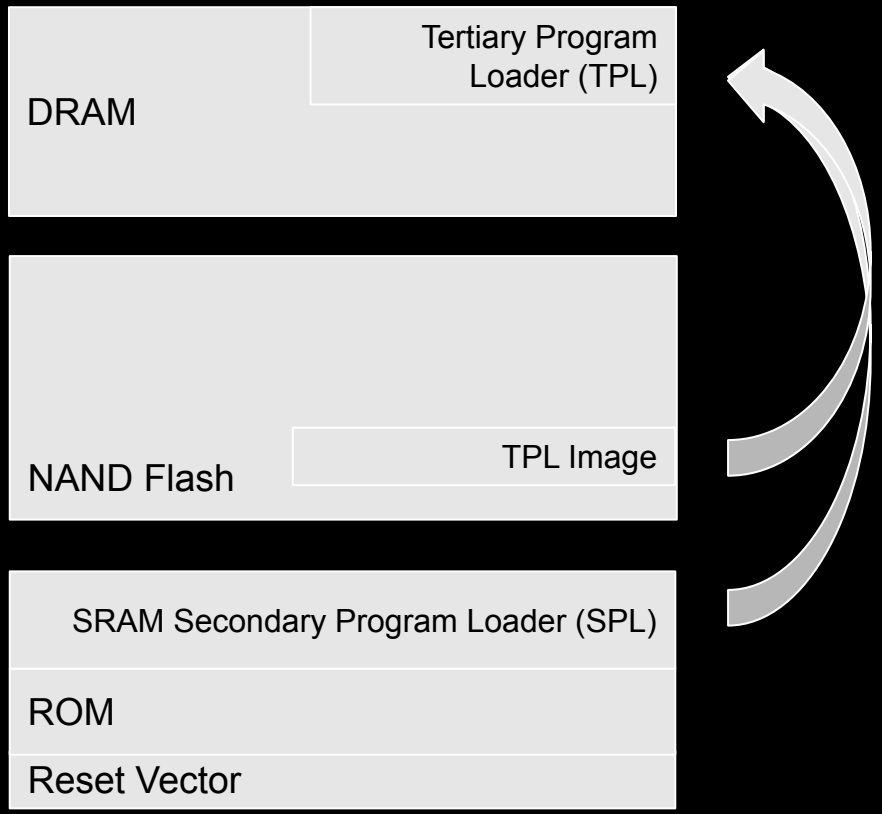
More Complicated Example



Phase 1: ROM code

- ROM initializes SPL (small SRAM area)
  - Default content from NAND
    - special section without ECC requirements
  - Transfer from external interface like USB (production)
- Reset vector boots into ROM

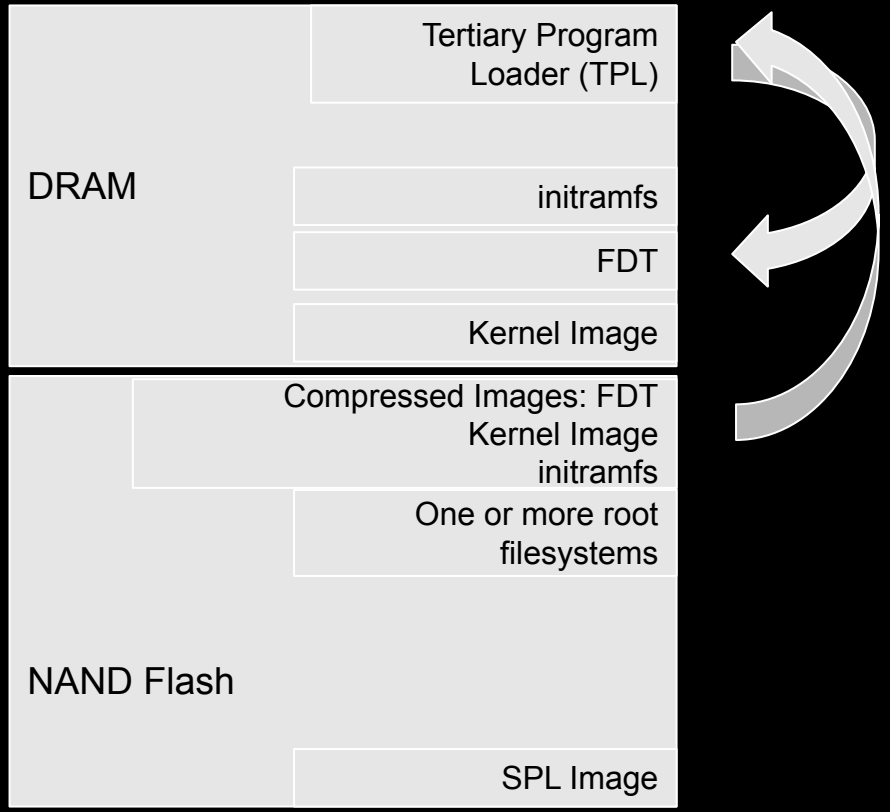
More Complicated Example



Phase 2: Secondary Program Loader

- SPL Initializes DRAM, then loads TPL from one of of:
  - Content received over USB or external interface (production)
  - Content from NAND flash (may have minimal ECC capabilities)

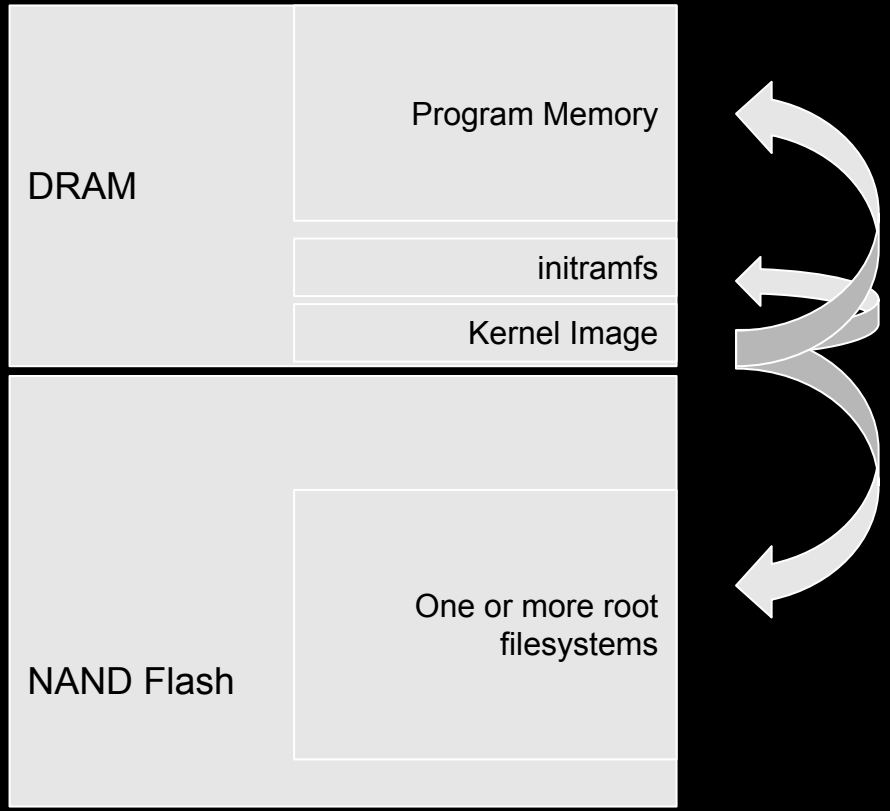
More Complicated Example



Phase 3: Tertiary Program Loader

- TPL (bootloader like U-boot) has full support for reading/writing NAND flash
- Reads/decompresses FDT, kernel, initramfs from NAND flash and places in DRAM
- Jumps into DRAM kernel image

More Complicated Example



Phase 4: Kernel Boot

- Kernel initializes memory in DRAM.
- initramfs completes hardware initialization, sets up root filesystem.
- Kicks off the init process based on the content of the root filesystem.

Bootloaders and Device Trees part 2

Advanced Embedded Software Development

with **Dan Walkes**


University of Colorado **Boulder**

Learning objectives:

Introduce bootloader options

Introduce U-Boot and UEFI

Understand basic device tree modifications for Raspberry Pi and Beaglebone

UEFI Firmware

- Universal Extensible Firmware Interface http://www.uefi.org/
- Common on embedded x86/x86_64 designs, also on some ARM designs
- Boot firmware BIOS replacement
- Uses a 3 phase bootloader as described in the previous video.

Bootloader Options

Name	Main architectures supported
Das U-Boot	ARC, ARM, Blackfin, Microblaze, MIPS, Nios2, OpenRiec, PowerPC, SH
Barebox	ARM, Blackfin, MIPS, Nios2, PowerPC
GRUB 2	X86, X86_64
Little Kernel	ARM
RedBoot	ARM, MIPS, PowerPC, SH
CFE	Broadcom MIPS
YAMON	MIPS

- Does it support your platform?
- Is it provided with your BSP?
- Is source available?
- Can you get support if needed?

U-Boot

- Started as an open source bootloader for PowerPC, then ported to ARM
- Very common on ARM platforms today
- Built using defconfig/kconfig structure used by the Linux kernel. make [board]_defconfig
- Builds uboot.bin/img files suitable for running on a device or programming into flash
- Depending on architecture, may also build a Secondary Program Loader (SPL), named MLO

Installing U-Boot

- Typically requires JTAG and/or hardware specific instructions.
- May be possible to load directly into RAM with JTAG, then bootstrap write into flash

Using U-Boot

- Use serial terminal to access.
- Includes a command prompt
  - help to list commands
- Uses environment variables to pass information or script commands
  - Initially defined by header files for the platform

Using U-Boot

- Set environment variables with setenv
- Save environment variables with saveenv
- Commands to load images into RAM from flash or tftp
- Commands to boot kernel image from RAM
  - bootm [address of kernel] [address of ramdisk] [address of dtb]
  - Assumes all have been loaded in memory using process like the one described previously.
- bootcmd is a variable run at power up to script a sequence of platform specific commands.

Raspberry Pi and DeviceTree

The current implementation is not a pure Device Tree system – there is still board support code that creates some platform devices – but the external interfaces (I2C, I2S, SPI), and the audio devices that use them, must now be instantiated using a Device Tree Blob (DTB) passed to the kernel by the loader (`start.elf`).

The main impact of using Device Tree is to change from **everything on**, relying on module blacklisting to manage contention, to **everything off unless requested by the DTB**. In order to continue to use external interfaces and the peripherals that attach to them, you will need to add some new settings to your `config.txt` . Full details are given in Part 3, but these are a few examples:

Mastering Embedded Linux Programming Chapter 3
Lecture 7
https://github.com/cu-ecen-5013/buildroot-assignments-base/wiki/Raspberry-Pi-Hardware-Hints
https://www.raspberrypi.org/documentation/configuration/device-tree.md

Raspberry Pi and DeviceTree

The Raspberry Pi uses a configuration file instead of the BIOS you would expect to find on a conventional PC. The system configuration parameters, which would traditionally be edited and stored using a BIOS, are stored instead in an optional text file named `config.txt`. This is read by the GPU before the ARM CPU and

As described above, DT parameters are a convenient way to make small changes to a device's configuration. The current base DTBs support parameters for enabling and controlling the onboard audio, I2C, I2S and SPI interfaces without using dedicated overlays. In use, parameters look like this:

```
dtparam=audio=on,i2c_arm=on,i2c_arm_baudrate=400000,spi=on
```

Mastering Embedded Linux Programming Chapter 3
Lecture 7
https://www.raspberrypi.org/documentation/configuration/config-txt/
https://www.raspberrypi.org/documentation/configuration/device-tree.md

Raspberry Pi and DeviceTree

Changes to config.txt can be performed either by accessing the first partition of the sdcard from another system, or you can mount the partition on a running system like this:

```
mkdir -p /tmp/mnt/p1
mount /dev/mmcblk0p1 /tmp/mnt/p1
```

You need to enable SPI in the device overlay by adding this line to config.txt:

```
dtparam=spi=on
```

https://github.com/cu-ecen-5013/buildroot-assignments-base/wiki/Raspberry-Pi-Hardware-Hints
https://www.raspberrypi.org/documentation/configuration/device-tree.md

Raspberry Pi and DeviceTree

```
dtc -I fs -O dts /sys/firmware/devicetree/base
```

Input = fs (/sys tree style directory)
Output = dts device tree source

After completing this step you should notice that your device tree shown with dtc looks like this:

```
spi@7e204000 {
        compatible = "brcm,bcm2835-spi";
        clocks = < 0x08 0x14 >;
        status = "okay";
```

instead of this:

```
spi@7e204000 {
        compatible = "brcm,bcm2835-spi";
        clocks = < 0x08 0x14 >;
        status = "disabled";
```

Mastering Embedded Linux Programming Chapter 3
https://github.com/cu-ecen-5013/buildroot-assignments-base/wiki/Raspberry-Pi-Hardware-Hints
https://www.raspberrypi.org/documentation/configuration/device-tree.md

Beaglebone and Device Tree

```
From 72401b2abab99007f02d1a036e9929241bd945be Mon Sep 17 00:00:00 2001
From: MohitRane8 <mohit.rane@colorado.edu>
Date: Sun, 15 Dec 2019 13:39:43 -0700
Subject: [PATCH] Added UART 1, 4 and 5 support for BeagleBone Black. Original
 Author: Matwey V. Kornilov, Edited by: Mohit Rane

---
 arch/arm/boot/dts/am335x-bone-common.dtsi | 42 ++++
 1 file changed, 42 insertions(+)

diff --git a/arch/arm/boot/dts/am335x-bone-common.dtsi b/arch/arm/boot/dts/am335x-bone-common.dtsi
index e13aed0963..51a6be7ed5 100644
--- a/arch/arm/boot/dts/am335x-bone-common.dtsi
+++ b/arch/arm/boot/dts/am335x-bone-common.dtsi
```

```
+    uart1_pins: pinmux_uart1_pins {
+        pinctrl-single,pins = <
+            AM33XX_IOPAD(0x980, PIN_INPUT_PULLUP | MUX_MODE0)   /* uart1_rxd.uart1_rxd */
+            AM33XX_IOPAD(0x984, PIN_OUTPUT_PULLDOWN | MUX_MODE0)    /* uart1_txd.uart1_txd */
+        >;
+    };
```

```
+&uart1 {
+    pinctrl-names = "default";
+    pinctrl-0 = <&uart1_pins>;
+
+    status = "okay";
+};
```

Mastering Embedded Linux Programming Chapter 3

https://github.com/cu-ecen-5013/buildroot-assignments-base/wiki/Enabling-UART-on-BeagleBone-Black

https://github.com/cu-ecen-5013/buildroot-assignments-base/pull/4/files

https://github.com/MohitRane8/buildroot-assignments-base/blob/88a311bda67670a1ff841057676a211d5e4dce9d/base_external/patches/linux/0001-Added-UART-1-4-and-5-support-for-BeagleBone-Black.patch

https://github.com/devicetree-org/devicetree-specification

Beaglebone and Device Tree

```
BR2_LINUX_KERNEL_CUSTOM_REPO_URL="git://git.ti.com/processor-sdk/processor-sdk-linux.git"
BR2_LINUX_KERNEL_CUSTOM_REPO_VERSION="4796173fc58688055a99a1cef19a839174067220"
```

Look at buildroot configuration to find linux kernel source

```
/dts-v1/;

#include "am33xx.dtsi"
#include "am335x-bone-common.dtsi"

/ {
    model = "TI AM335x BeagleBone";
    compatible = "ti,am335x-bone", "ti,am33xx";
};
```

Common devicetree include file for the processor

The file we've patched in previous slide to include uart definitions

```
        uart0: serial@44e09000 {
            compatible = "ti,am3352-uart", "ti,omap3-uart";
            ti,hwmods = "uart1";
            clock-frequency = <48000000>;
            reg = <0x44e09000 0x2000>;
            interrupts = <72>;
            status = "disabled";
            dmas = <&edma 26 0>, <&edma 27 0>;
            dma-names = "tx", "rx";
        };

"arch/arm/boot/dts/am33xx.dtsi" 1144L, 26958C written
```

Definition we added in bone-common.dsti

From linux kernel source documentation/devicetree/bindings/arm/omap/omap.txt

```
Required properties:
- compatible: Every devices present in OMAP SoC should be in the
  form: "ti,XXX"
- ti,hwmods: list of hwmod names (ascii strings), that comes from the OMAP
  HW documentation, attached to a device. Must contain at least
  one hwmod.
```

https://git.ti.com/cgit/processor-sdk/processor-sdk-linux/tree/arch/arm/boot/dts/am33xx.dtsi?h=processor-sdk-linux-01.00.00

https://git.ti.com/cgit/processor-sdk/processor-sdk-linux/tree/Documentation/devicetree/bindings/arm/omap/omap.txt?h=processor-sdk-linux-01.00.00